Distributed Chat System

Real-time chat with distributed architecture: Node.js, Redis, NATS, Consul, Socket.IO.

Features

✅ Real-time messaging • Persistent sessions • Username uniqueness • Presence tracking • Message history • Horizontal scaling • Service discovery • Multi-room support • Auto-discovery • **Automatic dependency management**

Quick Start

Prerequisites

The script automatically checks for and offers to install missing dependencies:

- **podman** - Container runtime
- **curl** - HTTP client for API calls
- **python3** - For JSON parsing and cluster helpers
- **jq** - JSON processor
- **ip** (iproute2) - Network utilities

```
# Make scripts executable
chmod +x chat-system.sh setup-cluster.sh cluster_bridge.py
cluster_helper.py

# Run - dependencies will be checked automatically
./chat-system.sh start
```

PROF

If any dependencies are missing, you'll be prompted:

```
✗ Missing dependencies: podman curl jq
Would you like to install missing dependencies? [y/N]
```

Supported package managers: apt (Debian/Ubuntu), dnf (Fedora), yum (CentOS/RHEL), pacman (Arch), zypper (openSUSE)

Manual Installation (if needed)

```
# Debian/Ubuntu
sudo apt install podman curl python3 jq iproute2

# Fedora
sudo dnf install podman curl python3 jq iproute
```

```
# Arch Linux
sudo pacman -S podman curl python jq iproute2
```

Auto-Discovery (Recommended)

```
# Automatically discovers cluster and infrastructure
./chat-system.sh start --auto

# Start client
cd client-react && npm install && npm run dev
```

Access: http://localhost:5173

What happens:

- Checks if main cluster Consul is available (172.20.10.10:8500)
- Discovers existing Redis/NATS services
- Starts missing services as needed
- Registers with cluster for service discovery
- Falls back to standalone if no cluster found

Single Machine Standalone

```
./chat-system.sh start
cd client-react && npm install && npm run dev
```

Consul Cluster (Multi-Machine)

PROF

First node:

```
./chat-system.sh start --cluster --mode infrastructure
```

Additional nodes:

```
./chat-system.sh start --auto
```

Each Consul server runs Redis + NATS + Chat Node. NATS auto-clusters via Consul.

Setup (on each server, one at a time):

```
```

```
sudo ./setup-consul-chat-node.sh --auto-cluster
```

Firewall:

```
sudo ufw allow 3001/tcp 4222/tcp 6222/tcp 6379/tcp 8300:8302/tcp
8500/tcp 8600/tcp
```

Verify cluster:

```
# Check NATS cluster connections
curl http://localhost:8222/routez | jq '.routes'

# Check Consul service discovery
curl http://localhost:8500/v1/catalog/service/chat-nats | jq

# Test from another machine
curl http://FIRST_MACHINE_IP:8222/routez | jq '.routes'
```

Expected output:

- `routez` shows connected peer IPs in `routes` array
- Consul shows all registered `chat-nats` services
- Each node reports `num_routes` > 0 (except first node initially)

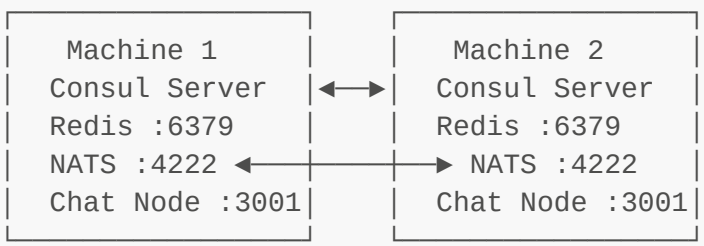
Architecture

Single Machine:

PROF

```
Client → Chat Nodes (3) → Redis + NATS + Consul
```

Distributed (each machine):



Messages sync across nodes via NATS cluster.

Testing Multi-Machine Setup

1. First Machine (Host)

```
# Check Consul is running
systemctl status consul

# Install chat stack
sudo ./setup-consul-chat-node.sh --auto-cluster

# Verify services
systemctl status chat-redis chat-nats chat-node

# Get your IP
ip addr show | grep 'inet ' | grep -v '127.0.0.1'

# Check NATS (should show no routes yet)
curl http://localhost:8222/routez | jq '.routes'
```

2. Second Machine (VM)

```
# Ensure Consul is clustered with first machine
consul members  # Should show both machines

# Install chat stack
sudo ./setup-consul-chat-node.sh --auto-cluster

# Verify services
systemctl status chat-redis chat-nats chat-node

# Check NATS cluster (should show connection to first machine)
curl http://localhost:8222/routez | jq
```

PROF

3. Verify Connection

On either machine:

```
# Check NATS cluster connections
curl http://localhost:8222/routez | jq '.routes'
# Expected: Array with peer connection info

# Check Consul service discovery
curl http://localhost:8500/v1/catalog/service/chat-nats | jq '.
[].Address'
# Expected: Both machine IPs listed

# View service logs
```

```
journalctl -u chat-node -f
# Expected: "Connected to NATS" message

# Test message sync (watch logs on both machines)
# Send message from one machine's client
# Should appear in logs on both machines
```

4. Test Chat Application

Machine 1:

```
cd client-react && npm install && npm run dev
# Open http://MACHINE1_IP:5173
# Login as "user1"
```

Machine 2:

```
cd client-react && npm install && npm run dev
# Open http://MACHINE2_IP:5173
# Login as "user2"
```

Expected behavior:

- user1 sees user2 join (presence event)
- Messages from user1 appear for user2 (and vice versa)
- Both see same message history

Success Indicators

PROF

✅ **NATS Cluster:** `routes` array not empty
✅ **Consul Registry:** Both IPs in service list
✅ **Message Sync:** Logs show same messages on both machines
✅ **Presence Sync:** Users see each other join/leave
✅ **No Errors:** `journalctl -u chat-node` shows no connection errors

Troubleshooting

NATS not clustering:

```
# Check firewall
sudo ufw status | grep 6222

# Test connectivity
nc -zv OTHER_MACHINE_IP 6222
```

```
# Restart NATS to retry connection
sudo systemctl restart chat-nats
sleep 5
curl http://localhost:8222/routez | jq '.routes'
```

Services not appearing in Consul:

```
# Check Consul connectivity
curl http://localhost:8500/v1/agent/self | jq '.Config.Datacenter'

# Re-register services
curl -X PUT http://localhost:8500/v1/agent/service/register -d
@/tmp/service.json
```

Messages not syncing:

```
# Check NATS connection in logs
journalctl -u chat-node -n 100 | grep -i nats

# Test NATS pub/sub manually
podman exec chat-nats nats pub test "hello"
# On other machine:
podman exec chat-nats nats sub test
```

Cluster Integration

How It Works

1. **Auto-Discovery**: Checks for main cluster Consul at `http://172.20.10.10:8500`
2. **Infrastructure Discovery**: Finds existing `redis-service` and `nats-service`
3. **Self-Registration**: Registers as `chat-service` with unique ID
4. **Peer Discovery**: Discovers other chat nodes automatically
5. **Unified Service**: External services see one `chat-service`, cluster handles load balancing

PROF

Service Discovery (Other Services)

```
from cluster_helper import pick_service_instance

# Get a chat node
address, port = pick_service_instance("chat-service")
response = requests.post(f"http://{address}:{port}/api/notify", json=
{...})
```

Autonomous Deployment

Each node autonomously:

- Discovers cluster infrastructure
- Starts missing services
- Registers itself
- Connects to peers
- **No manual IP configuration needed**

Management

Commands

```
./chat-system.sh start [--auto]        # Start with auto-discovery
(checks dependencies)
./chat-system.sh start                 # Standalone mode (checks
dependencies)
./chat-system.sh stop                  # Stop all services
./chat-system.sh restart               # Restart chat nodes
./chat-system.sh status                # Show status
./chat-system.sh logs <service>        # View logs (redis, nats, consul,
chat-1/2/3)
./chat-system.sh build                 # Rebuild Docker image (checks
dependencies)
./chat-system.sh clear-usernames       # Clear all registered usernames
from Redis
./chat-system.sh cluster-test [url]    # Test connectivity to cluster
Consul
./chat-system.sh deregister [url]      # Manually deregister services
from cluster
./chat-system.sh help                  # Show all options
```

Options

PROF

- `--auto` - Auto-discover cluster (recommended)
- `--cluster` - Force cluster mode
- `--cluster-consul <URL>` - Cluster Consul URL (default: http://172.20.10.10:8500)
- `--mode <mode>` - `standalone` | `infrastructure` | `node-only` | `auto`

Scaling

```
# Add a new node (automatically discovers and connects)
./chat-system.sh start --auto

# Remove a node (gracefully deregisters)
./chat-system.sh stop
```

Development

Backend

```
cd chat-node
npm install && npm run build
sudo podman build -t chat-node:latest .
```

Structure: `src/gateway.ts` (Socket.IO) • `src/chatcore.ts` (Redis→NATS) • `src/redis.ts` • `src/nats.ts` • `src/consul.ts`

Frontend

```
cd client-react
npm install && npm run dev
```

Structure: `src/pages/chatPage.tsx` • `src/components/` • `src/hooks/useSocket.tsx`

Technical Details

Message Flow

1. Client sends → Socket.IO (gateway.ts)
2. Write to local Redis Stream (persistence)
3. Publish to local NATS (real-time)
4. NATS cluster syncs to all nodes
5. All nodes receive → broadcast to clients

Data Storage

PROF

- **Redis Streams:** Message history (per room)
- **Redis Keys:** Username registry (case-insensitive, TTL 3600s)
- **NATS:** Ephemeral pub/sub (no storage)

Load Balancing

Client randomly selects node on connect and automatically fails over to other nodes if connection fails. Configure multiple nodes for high availability.

Health Checks

- Chat nodes: `GET /health` → `{"status":"healthy","nodeId":"..."}`
- Consul polls every 10s

Configuration

Environment Variables (systemd services)

- `NODE_ID` - Hostname
- `PORT` - 3001 (default)
- `REDIS_URL` - redis://localhost:6379
- `NATS_URL` - nats://localhost:4222
- `CONSUL_URL` - http://localhost:8500

Client Configuration

The React client supports automatic failover between multiple chat nodes.

Create `client-react/.env` file:

```
# Local development (default)
VITE_CHAT_NODE_URL=http://localhost:3001,http://localhost:3002,http://lo
calhost:3003

# Remote nodes (single host)
VITE_CHAT_NODE_URL=http://192.168.100.231:3001,http://192.168.100.231:30
02,http://192.168.100.231:3003

# Multiple hosts (high availability)
VITE_CHAT_NODE_URL=http://192.168.100.231:3001,http://192.168.100.162:30
01,http://192.168.100.53:3001
```

How it works:

1. Client randomly selects one node on startup (load balancing)
2. If connection fails after 3 attempts, automatically tries next node
3. Continues cycling through nodes until successful connection
4. Shows which node you're connected to in the UI

PROF

No configuration needed - defaults to localhost:3001-3003 if `.env` not present.

API Reference

Client → Server:

- `setUsername(string)` - Register
- `join({roomId})` - Join room
- `message({roomId, message})` - Send
- `typing({roomId, isTyping})` - Typing indicator

Server → Client:

- `usernameAccepted({username})` - Auth OK
- `joined({roomId})` - Room joined
- `history(Message[])` - Load history

- `message(Message)` - New message
- `userJoined/Left({username})` - Presence
- `userTyping({username})` - Typing

Project Structure

```
chat-system-test/
├── chat-system.sh                    # Single-machine manager
├── setup-consul-chat-node.sh         # Consul cluster installer
├── README.md
├── chat-node/                        # Backend (Node.js)
│   ├── src/{gateway,chatcore,redis,nats,consul}.ts
│   └── Dockerfile
└── client-react/                     # Frontend (React)
    └── src/{pages,components,hooks}/
```

Monitoring and Metrics

Metrics Endpoint

Each chat node exposes Prometheus-compatible metrics at `/metrics`:

```
# From local nodes
curl http://localhost:3001/metrics
curl http://localhost:3002/metrics
curl http://localhost:3003/metrics

# From remote nodes
curl http://192.168.x.x:3001/metrics
```

PROF

Available Metrics:

- `chat_connections_total` - Active WebSocket connections per node
- `chat_redis_circuit_breaker` - Redis circuit breaker state (0=closed, 1=open, 2=half-open)
- `chat_nats_circuit_breaker` - NATS circuit breaker state
- `chat_health_status` - Overall health (0=unhealthy, 1=healthy)

Aggregate Metrics Across All Nodes

The metrics aggregator automatically discovers all chat instances via Consul:

```
# Create the aggregator script
cat > metrics_aggregator.py << 'EOF'
#!/usr/bin/env python3
import sys, requests
```

```
def aggregate_metrics(consul_url="http://192.168.100.53:8500"):
    try:
        response = requests.get(f"{consul_url}/v1/health/service/chat-
service?passing=true", timeout=5)
        response.raise_for_status()
        instances = response.json()

        if not instances:
            print("# No healthy chat-service instances found")
            return

        total_connections = 0
        instance_metrics = []

        for entry in instances:
            svc = entry['Service']
            service_id, address, port = svc['ID'], svc['Address'],
svc['Port']
            try:
                metrics_response = requests.get(f"http://{address}:
{port}/metrics", timeout=2)
                if metrics_response.status_code == 200:
                    instance_metrics.append(metrics_response.text)
                    for line in metrics_response.text.split('\n'):
                        if line.startswith('chat_connections_total'):
                            total_connections += int(line.split()[-1])
            except Exception as e:
                print(f"# Warning: Could not fetch from {service_id}:
{e}")

        print(f"# HELP chat_total_connections_aggregate Total
connections across all nodes")
        print(f"# TYPE chat_total_connections_aggregate gauge")
        print(f"chat_total_connections_aggregate {total_connections}\n")
        print(f"# TYPE chat_instances_total gauge")
        print(f"chat_instances_total {len(instances)}\n")
        for metrics in instance_metrics:
            print(metrics + "\n")
    except Exception as e:
        print(f"# Error: {e}", file=sys.stderr)
        sys.exit(1)

if __name__ == '__main__':
    consul_url = sys.argv[1] if len(sys.argv) > 1 else
"http://192.168.100.53:8500"
    aggregate_metrics(consul_url)
EOF

chmod +x metrics_aggregator.py

# Run it
python3 metrics_aggregator.py http://192.168.100.53:8500
```

Prometheus Integration

Scrape Configuration:

```
scrape_configs:
  - job_name: 'chat-system'
    consul_sd_configs:
      - server: '192.168.100.53:8500'
        services: ['chat-service']
    relabel_configs:
      - source_labels: [__meta_consul_service_address]
        target_label: __address__
        replacement: '$1:__meta_consul_service_port__'
    metrics_path: '/metrics'
```

Troubleshooting

Quick Diagnostic Script

Use this comprehensive diagnostic tool to check registration and connectivity:

```
cat > diagnose-registration.sh << 'DIAGEOF'
#!/bin/bash
CONSUL_IP=${1:-"192.168.100.53"}
CONSUL_URL="http://${CONSUL_IP}:8500"

echo "=== Chat System Registration Diagnostics ==="
echo "Consul Server: $CONSUL_URL"

# Test Consul connectivity
echo -e "\n1. Testing Consul connectivity..."
if curl -sf "$CONSUL_URL/v1/status/leader" > /dev/null 2>&1; then
    echo "   ✓ Consul is reachable"
else
    echo "   x Cannot reach Consul - check firewall/IP"
    exit 1
fi

# Check running containers
echo -e "\n2. Checking running chat-node containers..."
CONTAINERS=$(sudo podman ps --filter "name=chat-node" --format "
{{.Names}}")
if [ -z "$CONTAINERS" ]; then
    echo "   x No chat-node containers running"
    exit 1
fi
for container in $CONTAINERS; do echo "   - $container"; done
```

```
# Check environment variables
echo -e "\n3. Checking container environment variables..."
for container in $CONTAINERS; do
    echo "   === $container ==="
    sudo podman exec "$container" env | grep -E
"CONSUL_URL|SERVICE_ID|HOST_IP|PORT|CLUSTER_MODE" | sort
done

# Test Consul connectivity from containers
echo -e "\n4. Testing Consul connectivity from containers..."
for container in $CONTAINERS; do
    echo -n "   $container: "
    if sudo podman exec "$container" curl -sf
"$CONSUL_URL/v1/status/leader" > /dev/null 2>&1; then
        echo "✓ Can reach Consul"
    else
        echo "✗ Cannot reach Consul"
    fi
done

# Check container logs
echo -e "\n5. Checking container logs for registration..."
for container in $CONTAINERS; do
    echo "   === $container ==="
    sudo podman logs --tail 30 "$container" 2>&1 | grep -E
"Registering|Registered|Failed"
done

# Check Consul registry
echo -e "\n6. Services registered in Consul..."
echo "   === chat-service instances ==="
curl -s "$CONSUL_URL/v1/catalog/service/chat-service" | jq -r '.[] | "
- \(.ServiceID) @ \(.ServiceAddress):\(.ServicePort)"'
echo "   === redis-service instances ==="
curl -s "$CONSUL_URL/v1/catalog/service/redis-service" | jq -r '.[] | "
- \(.ServiceID) @ \(.ServiceAddress):\(.ServicePort)"'
echo "   === nats-service instances ==="
curl -s "$CONSUL_URL/v1/catalog/service/nats-service" | jq -r '.[] | "
- \(.ServiceID) @ \(.ServiceAddress):\(.ServicePort)"'

# Check health
echo -e "\n7. Health status..."
curl -s "$CONSUL_URL/v1/health/state/passing" | jq -r '.[] |
select(.ServiceName | test("chat|redis|nats")) | "   ✓ \(.ServiceID)"'
curl -s "$CONSUL_URL/v1/health/state/critical" | jq -r '.[] |
select(.ServiceName | test("chat|redis|nats")) | "   ✗ \(.ServiceID): \
(.Output)"'

echo -e "\n=== Expected Service IDs ==="
echo "chat-node-192-168-X-X-1, redis-192-168-X-X, nats-192-168-X-X"
echo -e "\nIf services missing: ./chat-system.sh build && ./chat-
system.sh start --cluster --cluster-consul $CONSUL_URL"
DIAGEOF
```

```
chmod +x diagnose-registration.sh

# Run diagnostics
./diagnose-registration.sh 192.168.100.53
```

Common Issues

Dependencies not installing automatically

```
# Check your package manager
which apt-get dnf yum pacman zypper

# Manual install (Debian/Ubuntu)
sudo apt update && sudo apt install -y podman curl python3 jq iproute2

# Manual install (Fedora)
sudo dnf install -y podman curl python3 jq iproute
```

Registration Issues: Services Not Appearing in Consul

Symptom: Multi-host nodes not showing up or duplicated IDs

Diagnostic Steps:

```
# 1. Rebuild image with latest registration code
./chat-system.sh build

# 2. Start with explicit cluster consul
./chat-system.sh start --cluster --cluster-consul
http://192.168.100.53:8500

# 3. Check container environment
sudo podman inspect --format '{{range .Config.Env}}{{println .}}{{end}}'
chat-node-1 | grep -E 'CONSUL_URL|SERVICE_ID|HOST_IP'

# Expected output:
# CONSUL_URL=http://192.168.100.53:8500
# SERVICE_ID=chat-node-192-168-x-x-1
# HOST_IP=192.168.x.x

# 4. View registration logs
sudo podman logs --tail 200 chat-node-1 | grep -E
"Registering|Registered"

# Look for:
# 📝 Registering chat-node-192-168-x-x-1 with Consul at
http://192.168.100.53:8500
```

```
# ✓ Registered chat-node-192-168-x-x-1 with cluster at 192.168.x.x:3001

# 5. Query Consul for registered services
curl http://192.168.100.53:8500/v1/catalog/service/chat-service | jq -r
'.[] | "\(.ServiceID) \(.Address):\(.ServicePort)"'
```

Common Fixes:

- **Services registering with localhost/127.0.0.1:**

```
# Verify host IP detection
ip addr show | grep 'inet ' | grep -v '127.0.0.1'

# Override if needed
export HOST_IP=192.168.x.x
./chat-system.sh start --cluster --cluster-consul
http://192.168.100.53:8500
```

- **Cannot reach Consul at 192.168.100.53:8500:**

```
# Check firewall
sudo ufw status | grep 8500

# Test connectivity
ping 192.168.100.53
curl http://192.168.100.53:8500/v1/agent/self

# Use SSH tunnel if needed
ssh -L 8500:192.168.100.53:8500 user@consul-host
./chat-system.sh start --cluster --cluster-consul
http://localhost:8500
```

PROF

- **Duplicate service IDs:**

```
# Service IDs should be unique per host: chat-node-<host-ip>-
<index>
curl http://192.168.100.53:8500/v1/catalog/service/chat-service |
jq -r '.[].ServiceID'

# Expected for 2 hosts:
# chat-node-192-168-1-100-1
# chat-node-192-168-1-101-1
```

Nodes not discovering each other

```
# Test cluster connectivity
curl http://172.20.10.10:8500/v1/agent/self

# Check if services are registered
curl http://172.20.10.10:8500/v1/catalog/service/chat-service

# View logs
./chat-system.sh logs chat-1
```

Services not starting

```
# Rebuild image (also checks dependencies)
./chat-system.sh build

# Check Python dependencies
pip3 install requests

# Verify cluster helper scripts exist
ls -la cluster_bridge.py cluster_helper.py
```

Health checks failing

```
# Test health endpoint
curl http://localhost:3001/health

# Check connectivity to infrastructure
redis-cli -h <redis-host> PING
curl http://<nats-host>:4222
```

PROF

Container cannot reach Consul

```
# Test from inside container
sudo podman exec -it chat-node-1 sh
wget -O- http://192.168.100.53:8500/v1/agent/self

# Verify container uses host network
sudo podman inspect chat-node-1 | jq '.[].HostConfig.NetworkMode'
# Should show: "host"
```

Manual Consul Registration Test

```
```

```
# Test if you can manually register a service
curl -X PUT http://192.168.100.53:8500/v1/agent/service/register \
  -d '{
    "ID": "test-service",
    "Name": "test",
    "Address": "192.168.x.x",
    "Port": 9999
  }'

# Verify it appears
curl http://192.168.100.53:8500/v1/catalog/service/test

# Cleanup
curl -X PUT http://192.168.100.53:8500/v1/agent/service/deregister/test-
service
```

Performance

- **Messages/sec:** ~1000/node
- **Concurrent users:** ~500/node
- **Latency:** <50ms (local network)

License

MIT